

Project 01: The Searchin? Pac-Man

CSE 537 - Artificial Intelligence

Search implementation and performance analysis

Param Shah: 117369485

Madhura Chitale : 117420795

1. Implementation and evaluation method

We implemented Questions 1 to 7 in `search.py` and `searchAgents.py`. The implementation covered depth-first search (DFS), breadth-first search (BFS), uniform-cost search (UCS), A*, the corners problem, and heuristics for visiting corners and collecting food. We retained the supplied function and class names, framework code, expansion counters, and Berkeley attribution.

Search implementation

DFS used a stack and recorded expanded states. BFS used a FIFO queue and recorded states when they were first enqueued. UCS reused A* with a zero heuristic, which made its priority equal to accumulated path cost. A* prioritized entries by $g+h$, tracked the cheapest known route to each state, skipped outdated entries, and allowed reopening when a cheaper route was discovered. Each algorithm returned a list of actions.

Runtime, memory, and node measurements

The evaluation covered 28 benchmark cases. Each reported runtime was the median of three fresh searches measured with `time.perf_counter`. Each run began with a new problem and empty heuristic caches, and garbage collection ran before timing. Imports, layout loading, problem construction, graphics, and plan execution were excluded.

Peak memory was measured in a separate fresh run with `tracemalloc`. It included search allocations, bookkeeping, and heuristic caches, but excluded pre-existing interpreter and maze allocations. The values represented traced Python memory rather than total process memory. One KiB corresponded to 1,024 bytes.

Node counts were read from `problem._expanded` before replaying the returned plan. Auxiliary BFS calculations inside the food heuristic contributed to runtime and memory but were not included in that expansion counter. All 28 plans were replayed to check that their moves were legal and that they reached the intended goal.

Tables 1-4 present the complete benchmark results. Steps and path cost were equal for unit-cost problems. The weighted costs were reported separately below Table 2. A bounded `mediumSearch` trial was discussed separately because it did not produce a completed result. Python 3.11, which the assignment recommended, was not available in the measured environment.

2. Questions 1 &2: DFS and BFS

Table 1 shows the fixed-position results for all four search strategies. BFS, UCS, and Manhattan A* returned equal shortest-path lengths on these unit-cost mazes. DFS returned longer routes on tinyMaze, mediumMaze, and openMaze.

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyMaze	DFS	10	15	0.074	3.93
tinyMaze	BFS	8	15	0.071	3.85
tinyMaze	UCS	8	15	0.077	4.20
tinyMaze	A* Manhattan	8	14	0.083	4.22
mediumMaze	DFS	130	146	0.581	30.89
mediumMaze	BFS	68	269	0.974	38.95
mediumMaze	UCS	68	269	1.106	41.08
mediumMaze	A* Manhattan	68	221	0.997	38.55
bigMaze	DFS	210	390	1.581	116.82
bigMaze	BFS	210	620	2.452	103.26
bigMaze	UCS	210	620	2.657	90.11
bigMaze	A* Manhattan	210	549	2.714	88.81
openMaze	DFS	298	576	3.229	562.52
openMaze	BFS	54	682	2.642	99.49
openMaze	UCS	54	682	3.106	123.12
openMaze	A* Manhattan	54	535	2.466	85.63

Table 1. Fixed-position search performance on unit-cost mazes.

DFS exploration order and actual movement

The DFS implementation followed the expected LIFO order. It pushed legal successors in North, South, East, West order and processed the last eligible successor first. It continued deeply along a branch before processing alternatives and skipped previously expanded states. This explanation was based on the implementation; the graphical overlay was not separately inspected.

Pac-Man did not need to traverse every expanded square. Expansion represented planning, while execution followed only the returned route. As Table 1 shows, DFS expanded 146 states on mediumMaze but returned 130 moves. Those numbers measured different aspects of the search.

Least-cost solutions

DFS did not guarantee minimum cost: its mediumMaze route required 130 moves, while BFS required 68. The algorithm prioritized depth and stopped at its first selected goal. Its behavior depended on successor order. The 130-step result differed from the assignment's approximate examples even though the supplied order was preserved; the cause of that discrepancy was not established.

BFS found least-cost routes on the tested unit-cost mazes because FIFO ordering explored increasing depth. Its discovered set prevented duplicate enqueueing and expansion. Equal step costs were essential to this conclusion; a fewest-step route would not necessarily minimize weighted cost. BFS, UCS, and A* also solved a

supplied one-move eight-puzzle instance without changes to search.py. The test initialized the module-level puzzle variable used by the supplied puzzle wrapper.

3. Questions 3 & 4: weighted UCS and A*

Weighted-cost search

Table 2 shows the UCS results for the two agents with exponential step costs. The same search algorithm handled both cases by ordering its frontier according to accumulated cost rather than the number of moves.

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
mediumDottedMaze	UCS east	74	186	0.987	41.12
mediumScaryMaze	UCS west	92	108	0.609	30.14

Table 2. UCS performance with exponential movement costs.

StayEastSearchAgent charged $0.5 \cdot x$ for entering column x , which made eastern positions cheaper. Its 74-move route had a weighted cost of 1.000976583804004. StayWestSearchAgent charged $2 \cdot x$, which made eastern positions more expensive. Its 92-move route had a weighted cost of 68,719,479,864. Both headless games ended in wins. These results explained the very low and very high costs described in Question 3.

A* on bigMaze

A* used $f=g+h$, where g was the accumulated cost and h was the supplied estimate of remaining cost. The Manhattan heuristic replaced the null default in the Question 4 test. Calling the same implementation with $h=0$ produced UCS behavior, so the shared code preserved both algorithms' priorities.

As Table 1 shows, Manhattan A* found the same 210-step solution as UCS on bigMaze and expanded 549 states instead of 620, matching the assignment's reference node counts. Its measured runtime was 2.714 ms, compared with 2.657 ms for UCS. Fewer expansions did not produce a faster run in this small measurement, where heuristic overhead and timing variation affected the result.

Comparison on openMaze

Table 1 also shows that DFS returned 298 moves on openMaze, whereas BFS, UCS, and A* each returned 54. BFS and UCS expanded 682 states, while A* expanded 535. A* took 2.466 ms and used 85.63 KiB, compared with 3.106 ms and 123.12 KiB for UCS.

DFS expanded fewer states than BFS on openMaze but used 562.52 KiB, compared with 99.49 KiB for BFS. This implementation stored complete action lists in frontier entries, so long paths and pending alternatives increased memory use. The result showed why node counts alone were insufficient for comparing search strategies.

4. Questions 5 & 6: visiting all corners

Table 3 shows the results for the corners problem. BFS and A* agreed on solution lengths, while the heuristic reduced the number of states expanded.

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyCorners	BFS	28	252	1.419	85.07
tinyCorners	A* corners	28	159	3.009	84.34
mediumCorners	BFS	106	1966	11.189	678.29
mediumCorners	A* corners	106	741	19.303	317.91

Table 3. BFS and A* performance on the corners problem.

State representation, successors, and goal test

Each CornersProblem state contained Pac-Man's position and an immutable tuple of four visited-corner flags. The representation omitted ghosts, score, and unrelated food. The problem object stored the fixed walls and corner coordinates. A starting corner was marked visited immediately, and immutable tuples kept alternative search branches independent.

Each legal move generated a new position and flag tuple at cost 1. Entering a corner set its flag, and previously set flags remained true. The goal test required all four flags to be true, regardless of whether food existed at those locations. As Table 3 shows, BFS found a 28-step route on tinyCorners and a 106-step route on mediumCorners. It expanded 1,966 states on mediumCorners, close to the assignment's reference of just under 2,000.

Corner heuristic and correctness

The heuristic computed the shortest Manhattan route through the unvisited corners, without requiring a return to the start. It considered all remaining visiting orders recursively, with at most $4! = 24$ complete orders. It selected the best overall order rather than greedily choosing the nearest next corner.

Removing walls produced a relaxed problem whose optimal cost could not exceed the real maze completion cost. This established admissibility. A legal move followed by an optimal relaxed route from its successor formed a candidate route from the original state, which established $h(s) \leq 1 + h(s')$. The heuristic was therefore consistent, nonnegative, and zero when all corners had been visited.

Performance interpretation

As Table 3 shows, A* reduced mediumCorners expansions from 1,966 to 741 and peak memory from 678.29 to 317.91 KiB while retaining the 106-step optimum. The node count was below the 1,200-node full-credit threshold and the 800-node stretch marker, which carried no additional stated points. Runtime increased from 11.189 to 19.303 ms, so heuristic evaluation outweighed the benefit of fewer expansions in this test.

5. Question 7: collecting all food

Table 4 shows the food-search comparison. A* and UCS returned equal solution costs on all three measured layouts, while the food heuristic substantially reduced search effort.

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
testSearch	UCS	7	14	0.419	16.75
testSearch	A* food MST	7	7	0.285	13.08
tinySearch	UCS	27	5057	219.103	8400.16
tinySearch	A* food MST	27	89	5.945	264.44
trickySearch	UCS	60	16688	1485.511	48296.31
trickySearch	A* food MST	60	255	30.424	1065.58

Table 4. UCS and A* performance on the food-search problem.

Food heuristic and caching

The supplied problem represented each state by position and a remaining-food grid. Successor generation copied the grid, removed food at the destination, and assigned cost 1 to each move. The heuristic added the shortest distance to the nearest remaining food to the minimum spanning tree (MST) weight over all remaining food locations.

BFS from each food location computed exact maze distances to reachable squares. These distance maps were cached because walls remained unchanged. Prim's algorithm computed the MST using those distances. Tree costs were cached by the remaining-food set, allowing different Pac-Man positions to reuse the same value. The MST supplied a lower bound rather than an executable route.

Results and grading thresholds

As Table 4 shows, both algorithms returned costs of 7 on testSearch, 27 on tinySearch, and 60 on trickySearch. On trickySearch, A* expanded 255 states instead of 16,688. Runtime decreased from 1,485.511 to 30.424 ms, and peak memory decreased from 48,296.31 to 1,065.58 KiB.

The assignment's node thresholds awarded 5 points below 15,000 expansions, 15 below 12,000, 20 below 9,000, and 5 extra points below 7,000, subject to heuristic correctness. The 255-node result met the highest listed threshold. No official autograder score was obtained.

Could mediumSearch be solved in a short time?

A separate headless AStarFoodSearchAgent trial did not finish within a 30-second limit and was terminated. It therefore did not demonstrate a short-time solution, and no final solution cost or expansion count was obtained. The assignment asked about this layout but did not specify a time limit or a separate scoring threshold. The 30-second limit was chosen for this trial and was not an assignment requirement.

6. Heuristic validation and critical analysis

Why the food heuristic was admissible

Let $d(p,F)$ denote the nearest-food distance from position p to remaining food F , and let $T(F)$ denote its MST weight. Every completion first had to reach a food dot at a cost of at least $d(p,F)$. The subsequent first-visit order defined a spanning path whose maze-distance weight did not exceed the remaining travel. The MST cost no more than that path. Thus $h(p,F)=d(p,F)+T(F)$ bounded successive portions of the route without overestimating the total completion cost.

Why the food heuristic was consistent

When a move did not eat food, the MST term remained unchanged and nearest-food distance could decrease by at most one. When a move ate dot a and left food R , attaching a to its nearest dot in an MST of R gave $T(F) \leq d(a,R)+T(R)$. The old nearest-food distance was at most one, so $h(p,F) \leq 1+h(a,R)$. Eating the last dot changed the heuristic from one to zero. Both cases therefore satisfied consistency.

Consistency established nondecreasing f -values along a path because g increased by the move cost. If UCS and A^* had returned different solution costs, the heuristic, implementation, and comparison setup would have required investigation. Matching costs alone would not have proved admissibility; the argument above supplied that justification.

Validation and limitations

Exhaustive checks verified consistency across 912 transitions over 384 tinyCorners states and 17,732 transitions over 8,100 tinySearch states. Checked values were nonnegative and zero at goals. Additional checks covered start-at-goal, unreachable goals, cycles, cheaper duplicate routes, A^* reopening, starting at a corner, and parent-state independence. Source compilation and representative headless games passed.

The package contained grading support code but lacked autograder.py and test_cases. Python 3.11 was also unavailable during validation. The measurements described the tested environment, and the incomplete mediumSearch trial limited conclusions about larger food problems.

Critical assessment

The results in Tables 1?4 showed that no single method performed best across every measured criterion. DFS sometimes expanded fewer states but produced longer routes and could consume substantial memory. BFS provided a reliable baseline for unit-cost shortest paths, while UCS handled nonuniform costs. Manhattan A^* reduced expansions, but its runtime benefit varied across layouts.

The corner heuristic reduced nodes and memory but ran more slowly than BFS. The food MST delivered the strongest improvement, retaining the optimal measured costs while sharply reducing nodes, runtime, and memory. These results supported comparing solution quality, expansion counts, runtime, and memory together rather than judging an algorithm by one statistic.